

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	HARI VISHNU N	Reg. No.:	192524319
Date:	07-10-2026	Duration:	60 minutes

Code of Conduct

I HARI VISHNU N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.

- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

PROBLEM STATEMENT:

(11) AI-Powered Requirements Conflict and Ambiguity Detection System

Industry Context:

Large software projects receive requirements from users, customers, business teams, developers, security teams, and regulators. Requirements may be incomplete, ambiguous, duplicated, or contradictory. These problems can remain hidden until design or implementation, resulting in costly rework.

Problem Statement:

Design and develop an AI-Powered Requirements Conflict and Ambiguity Detection System that analyses software requirements and identifies inconsistencies, ambiguity, duplication, missing dependencies, and conflicting stakeholder expectations. The system should support collaborative documentation, requirements prioritization, traceability, and human validation. It should integrate with project-management tools and maintain version history while ensuring that AI recommendations remain reviewable and explainable.

Objectives:

The system should:

- Analyse requirements for ambiguity and contradictions
- Detect duplicate and conflicting requirements
- Identify missing dependencies and incomplete information
- Support MoSCoW or similar prioritization techniques
- Maintain traceability between requirements and development tasks
- Provide collaborative review and version control
- Support explainable and human-validated AI recommendations

Expected Outcomes:

The proposed system will:

- Reduce requirements-related rework
- Improve clarity and consistency of software specifications
- Support better stakeholder collaboration
- Improve requirements traceability
- Reduce project risks caused by conflicting requirements
- Strengthen requirements engineering practices

SOLUTION:

1. Understand the Problem Statement

1.1 Problem statement:

In large-scale software development, requirements are gathered from a highly diverse set of stakeholders, including end-users, business analysts, developers, security teams, and regulatory bodies. Because these groups have different priorities and use different terminologies, the resulting documentation is frequently plagued by defects. Specifically, requirements may be duplicated across different modules, contradict one another (e.g., a security requirement conflicting with a user-convenience requirement), lack crucial technical details, or be phrased ambiguously.

The core issue is that these requirements defects often remain undetected during the initial planning phases. They are typically only discovered later during the active design or implementation (coding) phases. When developers encounter these hidden issues late in the Software Development Life Cycle (SDLC), it brings development to a halt and necessitates going back to the stakeholders for clarification, resulting in significant project delays and costly rework.

1.2 Objective:

The proposed "AI-Powered Requirements Conflict and Ambiguity Detection System" aims to proactively solve these issues through the following specific objectives:

- **Automated Linguistic Analysis:** Deploy AI to scan and analyze requirement texts to detect vague language, ambiguities, and logical contradictions before they reach the development team.
- **Conflict and Duplication Detection:** Automatically identify duplicate entries and conflicting expectations among different stakeholders to ensure a cohesive specification document.
- **Dependency Mapping:** Identify missing information and unmapped dependencies to ensure all prerequisites for a feature are fully documented.
- **Structured Prioritization:** Incorporate established frameworks like the MoSCoW method (Must-have, Should-have, Could-have, Won't-have) to help teams collaboratively prioritize valid requirements.
- **Comprehensive Traceability:** Maintain clear, trackable links between the initial requirements and the subsequent development tasks to ensure every requirement is accounted for.

- **Transparent and Collaborative Review:** Provide a centralized platform that supports version control, collaborative documentation, and human-validated reviews, ensuring that all AI-generated recommendations are explainable and transparent to the human users.

1.3 The scope of the problem:

The scope of this proposed system is strictly focused on the Requirements Engineering and Management phase of the software lifecycle.

- **In-Scope:** The system will process textual requirement inputs, analyze them using AI, flag anomalies, suggest MoSCoW prioritizations, and maintain version history. It will also feature integration capabilities to sync these validated requirements with standard project-management tools. It will act as an intelligent assistant to human analysts.
- **Out-of-Scope:** The system is not designed to automatically rewrite requirements without human intervention, nor is it intended to generate application code or software architectures. The scope explicitly mandates that all AI recommendations remain reviewable and subject to final human validation, meaning the system operates as a decision-support tool rather than an autonomous decision-maker.

2. Identify Key Stakeholders and Challenges:

2.1 Stakeholders:

In large-scale software engineering projects, requirements do not come from a single source. The system must account for inputs from users, customers, business teams, developers, security teams, and regulators. Furthermore, Project Managers and Quality Assurance (QA) teams are critical secondary stakeholders who rely on the outputs of the requirements engineering process.

2.2 Roles:

- **Users and Customers:** They are the domain experts who provide the initial business needs and operational goals. Their primary responsibility is to communicate their expectations and ultimately validate whether the software solves their real-world problems.
- **Business Teams (Business Analysts / Product Owners):** They act as the bridge between the clients and the technical teams. They are responsible for eliciting, writing, and organizing the requirements. In this proposed system, they will be the primary human reviewers responsible for managing collaborative documentation, applying MoSCoW prioritization, and validating the AI's recommendations.

- **Developers and Architects:** They are responsible for translating requirements into system design and executable code. They rely heavily on the completeness and clarity of the requirements to build the correct software architecture without having to guess the stakeholders' intent.
- **Security Teams:** They define the non-functional requirements regarding data protection, system integrity, and access controls. Their responsibility is to ensure the system is hardened against threats, which must be integrated seamlessly with the functional requirements.
- **Regulators:** They define the legal, compliance, and industry-specific mandates that the software must follow. Their requirements are non-negotiable and carry significant project risks if missed.
- **Project Managers:** They are responsible for overall project execution, risk management, and scheduling. They will utilize the system's integration with project-management tools to monitor traceability, track development tasks, and manage version history.

2.3 Major Challenges and Issues:

- **Hidden Contradictions Across Silos:** Because requirements are gathered from diverse groups, they frequently contradict one another. For example, a customer might demand "one-click instant login," while the security team mandates "multi-factor authentication." If these conflicting stakeholder expectations remain hidden until the design or implementation phases, development halts.
- **Ambiguity and Vague Terminology:** Customers and business teams often write requirements using subjective language (e.g., "The system must process data quickly"). This ambiguity leaves developers guessing. If a developer interprets the requirement incorrectly, it leads to costly rework later in the cycle.
- **Incomplete Information and Missing Dependencies:** In massive enterprise projects, functional requirements often lack the necessary underlying technical dependencies. If a requirement relies on an unlisted database or API, the implementation phase will be unexpectedly delayed when developers realize pieces are missing.
- **Duplication of Effort:** When hundreds of pages of requirements are gathered manually, the same feature is often requested multiple times in different ways. This duplication clutters the scope, makes accurate effort estimation impossible, and confuses the development team.
- **Traceability Breakdown:** Without intelligent tools, manually tracking the relationship between a specific requirement and its corresponding development task is incredibly difficult. When a

requirement is updated, failing to trace that change to the affected code introduces severe project risks.

- **The Cost of Late Discovery:** The overarching challenge is that manual reviews fail to catch these inconsistencies early. When requirements problems remain hidden until the design or implementation phases, it results in expensive and time-consuming rework, significantly inflating project budgets.

3. Analyze the Current Approach:

3.1 Existing System/Current Process:

In most traditional software engineering environments, the requirements elicitation process relies heavily on manual, document-centric workflows. Requirements are gathered from a highly varied set of stakeholders—including users, customers, business teams, developers, security teams, and regulators. This information is typically collected through siloed meetings, email threads, and stakeholder interviews. The findings are then manually compiled into static, monolithic documents like Microsoft Word specifications or Excel spreadsheets. Traceability is usually maintained through a manual Requirements Traceability Matrix (RTM), where business analysts attempt to map each text-based requirement to a corresponding system module or test case. Collaboration often involves emailing these documents back and forth, tracking changes manually, or pasting text blocks into isolated ticketing systems.

3.2 Strengths and Limitations:

- **Strengths:**
 - **Domain Expertise and Nuance:** The current manual process allows business analysts to apply deep human empathy and industry-specific context when interpreting customer needs.
 - **Universal Accessibility:** Static tools like Word, Excel, or PDF require zero technical training, making it easy for non-technical stakeholders (such as clients or legal regulators) to read and contribute.
 - **High Flexibility:** During the initial brainstorming phases, unstructured text documents allow for rapid, unconstrained capture of ideas.
- **Limitations:**
 - **Cognitive Overload and Human Error:** It is physically impossible for a human analyst to cross-reference thousands of lines of text to ensure absolute consistency. As a result,

incomplete, ambiguous, duplicated, or contradictory requirements easily slip through human reviews.

- **Siloed Perspectives:** Security teams, business teams, and developers often write requirements in isolation. The current system has no mechanism to automatically alert a business analyst if a newly added business rule violates an existing security protocol.
- **Static and Disconnected:** Traditional documents are static. When a requirement is altered, the current system does not automatically update dependencies or notify downstream developers, breaking the traceability chain.

3.3 Identified gaps:

- **The "Late Discovery" Gap:** The most severe flaw in the current approach is that requirement defects remain hidden until the design or implementation phases of the software project. Discovering a logical conflict while developers are actively writing code brings the project to a halt, directly resulting in costly rework, wasted developer hours, and schedule overruns.
- **Lack of Automated Semantic Analysis:** There is a complete absence of automated tools to analyze the actual *meaning* of the text. The current process cannot proactively detect duplicate entries, flag ambiguous phrasing, or identify missing technical dependencies across large datasets.
- **Traceability and Integration Gaps:** There is a critical gap between requirements documentation and actual project execution. The current manual process fails to maintain dynamic traceability between requirements and development tasks, and it does not seamlessly integrate with modern project-management tools.
- **Inefficient Prioritization and Version Control:** Existing methods lack a centralized, intelligent environment to support collaborative documentation, track version history, and systematically resolve conflicting stakeholder expectations using structured frameworks like MoSCoW prioritization.

4. Propose a Suitable Solution Using Software Engineering Principles:

4.1 Proposed Software Solution:

To address the severe issues of requirements remaining hidden until design or implementation, the proposed solution is a cloud-hosted "Intelligent Requirements Engineering Platform." This system will sit between the initial document drafting phase and the development phase. It will act as a centralized, web-based portal where stakeholders can upload static requirement documents or draft them directly within a

collaborative text editor. At its core, the system will utilize Natural Language Processing (NLP) and Large Language Models (LLMs) to automatically parse the text, semantically understand the context, and cross-reference every sentence against the entire project database. When the AI detects potential issues, it will not alter the text autonomously; instead, it will flag the text and provide explainable, human-validated AI recommendations, ensuring human oversight remains paramount.

4.2 Major Functional Modules:

The system architecture will be divided into several interconnected but distinct modules to handle specific tasks:

- **Document Ingestion and Parsing Module:** This module handles the upload of various file formats (Word, PDF, Excel) and breaks down large paragraphs into atomic, individual requirement statements for processing.
- **AI Analysis & NLP Engine:** This is the core computational module. It applies machine learning algorithms to analyze requirements for ambiguity, detect duplicate entries, and highlight contradictory statements across different stakeholder inputs. It also identifies incomplete information and missing technical dependencies.
- **Collaboration & Prioritization Workspace:** A user interface module that supports collaborative documentation among teams, allowing them to discuss flagged issues and systematically rank requirements using MoSCoW or similar prioritization techniques.
- **Traceability & Versioning Engine:** This module automatically maps relationships between different requirements to maintain traceability. It also manages strict version history so teams can track who changed what and why.
- **Integration API Gateway:** A dedicated pipeline designed to integrate with external project-management tools (like Jira, Azure DevOps, or Trello), allowing the finalized, conflict-free requirements to flow directly into developers' backlogs.

4.3 Functional and non-functional requirements

- **Functional Requirements (What the system must do):**
 - The system shall automatically analyze textual requirements to detect ambiguity and contradictions.
 - The system shall scan the database to detect duplicate and conflicting requirements from multiple stakeholders.

- The system shall identify missing dependencies and incomplete information within the specification documents.
- The system shall provide a framework to support MoSCoW prioritization techniques.
- The system shall dynamically maintain traceability links between the documented requirements and downstream development tasks.
- The system shall support collaborative review workflows and maintain a detailed version history.
- The system shall ensure that all AI-generated recommendations are explainable and require human validation.
- The system shall integrate seamlessly with existing project-management tools.
- **Non-Functional Requirements (How the system must perform):**
 - **Scalability:** The system must be capable of processing high volumes of requirements for large software projects without performance degradation.
 - **Performance:** The AI analysis must execute rapidly (e.g., parsing 100 pages of requirements in under 30 seconds) to avoid workflow bottlenecks.
 - **Security:** Since requirements often contain proprietary business logic, the system must encrypt data at rest and in transit, employing strict Role-Based Access Control (RBAC).
 - **Usability:** The AI interface must be highly intuitive, presenting complex linguistic conflicts in a clear, easy-to-understand visual format for non-technical stakeholders.

4.4 Application of Software Engineering Principles

- **Modularity:** The system uses a microservices architecture, separating the AI NLP engine from the front-end UI. This allows developers to upgrade the AI model without taking the entire application offline.
- **Maintainability & Reliability:** By supporting collaborative review and version control, the system ensures that changes can be rolled back safely, preventing data corruption and making the software highly reliable.
- **Explainability (AI Transparency):** By mandating that AI recommendations remain reviewable and explainable, the system adheres to modern ethical AI engineering principles, preventing a "black box" scenario where users blindly trust flawed machine logic.

5. Justification of the Chosen Software Development Methodology:

5.1 Selection of the Appropriate SDLC Framework:

For the design, development, and deployment of the AI-Powered Requirements Conflict and Ambiguity Detection System, the Agile software development framework, specifically utilizing the Scrum methodology, is selected.

5.2 Strategic and Technical Justification for the Agile Framework:

The selection of Agile over traditional sequential models (such as Waterfall) is driven by the high technical complexity and iterative nature of artificial intelligence and Natural Language Processing (NLP) solutions. The justification is grounded in the following technical alignments:

- **Iterative Machine Learning Calibration:** Developing an NLP engine capable of accurately analyzing requirements for ambiguity and contradictions requires continuous algorithmic tuning. Agile allows the engineering team to deploy initial language models, test them against real-world, contradictory stakeholder inputs, and refine the AI's accuracy in subsequent iterative cycles (Sprints). A rigid Waterfall approach would lock in a theoretical model early in the lifecycle, risking complete system failure if the AI underperforms in real-world semantic analysis.
- **Facilitating Human-in-the-Loop (HITL) Validation:** A core mandate of the proposed system is that AI recommendations remain reviewable and explainable, ensuring human validation. Agile thrives on continuous stakeholder engagement. By presenting functional increments of the software at the end of every Sprint, business teams and developers can continuously review the AI's output and validate that the system's explanations are intuitive, transparent, and accurate.
- **Adaptive Integration Capabilities:** The proposed system must seamlessly integrate with existing project-management tools. Because third-party application programming interfaces (APIs) are frequently updated by their vendors, an Agile approach provides the architectural flexibility to adapt the system's integration modules dynamically, mitigating the risk of broken toolchains during development.
- **Early Risk Mitigation in Prioritization:** The system must support collaborative review and MoSCoW prioritization techniques. Agile empowers stakeholders to test this collaborative interface early in the development cycle, ensuring that the UI effectively reduces conflicting stakeholder expectations before the full system is launched.

5.3 Support for Successful Project Development and Maintenance:

The Agile Scrum framework provides robust mechanisms for both the immediate execution of the project and its long-term lifecycle maintenance:

- **Optimized Project Execution (Development):**
 - **Incremental Value Delivery:** By partitioning development into two-to-four-week Sprints, the team delivers functional components incrementally. For instance, the document ingestion and version history modules can be finalized and tested while the more complex AI dependency-tracking module is still in development.
 - **Cross-Functional Collaboration:** Scrum enforces daily stand-up meetings and regular retrospectives, directly mirroring the system's own objective to support better stakeholder collaboration. This ensures development teams, data scientists, and business analysts remain perfectly aligned.
- **Sustainable Lifecycle Maintenance:**
 - **Continuous Integration and Continuous Deployment (CI/CD):** Agile naturally pairs with modern DevOps practices. Once the system is deployed, user feedback may reveal that the AI struggles to interpret specific regulatory jargon or missing dependencies unique to a new client. Agile processes allow the maintenance team to rapidly retrain the NLP model, test it, and deploy patches without disrupting the live collaborative environment.
 - **Scalability of Requirements Practices:** As the adopting organization grows, its requirements engineering practices will mature. The Agile framework ensures the system's architecture remains modular and adaptable, allowing future engineering teams to add new functionalities—such as advanced automated testing generation—without requiring a complete system overhaul.

6. Discuss Expected Outcomes and Limitations:

6.1 Benefits of the Proposed Solution

The deployment of the AI-Powered Requirements Conflict and Ambiguity Detection System will yield several high-impact benefits for the software development lifecycle:

- **Drastic Reduction in Costly Rework:** By identifying requirement defects during the initial planning phase, the system prevents errors from cascading into the design or implementation phases, thereby significantly reducing requirements-related rework and saving developer hours.
- **Enhanced Specification Quality:** The AI's semantic analysis will inherently improve the overall clarity and consistency of software specifications, ensuring that both technical and non-technical stakeholders share a unified understanding of the project goals.
- **Elevated Team Synergy:** By providing a centralized platform for collaborative documentation and structured review, the system will actively support better stakeholder collaboration across previously siloed departments (e.g., business teams, security, and developers).
- **Robust Project Tracking:** The automated mapping features will improve requirements traceability, giving project managers immediate visibility into how a specific business need translates into actionable code.
- **Strategic Risk Mitigation:** Ultimately, the platform will reduce overall project risks caused by conflicting requirements and fundamentally strengthen the organization's requirements engineering practices.

6.2. Identified Challenges

The proposed system directly targets the root causes of traditional requirements engineering failures:

- **Addressing Hidden Contradictions and Ambiguity:** Instead of relying on manual peer reviews where human fatigue leads to oversight, the system's NLP engine will systematically analyze requirements for ambiguity and contradictions in real-time. It proactively flags these issues before they can reach the development team.
- **Resolving Incomplete and Duplicated Efforts:** The system cross-references the entire database to immediately detect duplicate entries and identify missing dependencies or incomplete information, ensuring developers have all necessary prerequisites before coding begins.
- **Mitigating Conflicting Stakeholder Expectations:** Different departments often demand mutually exclusive features. The system addresses this by enforcing structured MoSCoW prioritization techniques (Must-have, Should-have, Could-have, Won't-have) within a shared workspace, forcing stakeholders to negotiate and align their expectations transparently.
- **Fixing Disconnected Workflows:** To solve the challenge of poor tracking, the system integrates directly with existing project-management tools and maintains a strict version history, ensuring continuous traceability between the approved requirements and the actual development tasks.

6.3. Potential Risks, Implementation Challenges, and Limitations

- **AI Limitations (False Positives/Negatives):** NLP models are not flawless. A major limitation is that the AI might flag highly specific, valid industry jargon as "ambiguous," creating false positives that waste analysts' time. Conversely, it might fail to detect deeply complex, multi-layered logical contradictions, giving teams a false sense of security.
- **The Risk of Automation Bias:** There is a significant risk that users will become overly reliant on the AI and blindly accept its suggestions. To combat this, the system must strictly enforce that all AI recommendations remain reviewable, explainable, and subject to human validation.
- **Implementation Challenges (Integration and Adoption):** Integrating a new AI system with legacy project-management tools can present technical hurdles due to mismatched API protocols. Furthermore, overcoming organizational resistance—convincing stakeholders to abandon familiar tools like Microsoft Word for a new, AI-driven collaborative dashboard—will require substantial change management and training.
- **Data Security Risks:** Analyzing software requirements means processing highly confidential, proprietary business data. If the AI model is not securely hosted on private, encrypted infrastructure, there is a risk of intellectual property leakage.

6.4. Suggest Possible Future Enhancements

To ensure the system scales with future software engineering advancements, the following enhancements could be implemented in subsequent iterations:

- **Automated Test-Case Generation:** Once the AI and human reviewers have finalized a clear, conflict-free set of requirements, a future module could automatically translate these specifications into baseline testing scripts for the Quality Assurance (QA) team.
- **Auto-Generation of Architectural Models:** The NLP engine could be upgraded to extract key entities, actors, and relationships from the text to automatically generate preliminary Unified Modeling Language (UML) diagrams (like Use Case or Class diagrams) for the software architects.
- **Predictive Effort and Cost Estimation:** By analyzing the complexity of the prioritized requirements and integrating with historical project data, the system could provide project managers with real-time, AI-driven estimates for development timelines and required budgets.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25